LUCID: an Updatable and Concurrent Learned Index for Larger-than-Memory Data Management

Chaohong Ma*, Xiaohui Yu†, Yifan Li†, Aishan Maoliniyazi‡, Xiaofeng Meng‡✉
*Hebei Normal University, China; †York University, Canada; ‡Renmin University of China, China
chaohma@hebtu.edu.cn, {xhyu, yifanli}@yorku.ca, {aishan, xfmeng}@ruc.edu.cn

***Abstract*—Learned indexes have shown great potential for improving data storage and access performance, but their adoption in real-world systems is limited when data exceed memory capacity. In-memory approaches become impractical at scale, while fully on-disk techniques often fail to meet the performance demands of diverse workloads. Hybrid storage architectures that combine memory's low latency with disk's high capacity offer a promising middle ground. However, deploying learned indexes in such environments, especially under dynamic and complex workloads, remains challenging. An effective solution must manage larger-than-memory datasets within constrained resources, support dynamic updates, and enable concurrent operations—requirements that existing methods do not satisfy simultaneously. This paper presents LUCID, an Updatable and Concurrent Learned Index for larger-than-memory Data management. LUCID is a learned tree structure that indexes data spanning memory and disk, offering robust support for dynamic and concurrent workloads. First, it employs model-guided buffering to optimize in-memory insertions. Second, it introduces an overflow strategy at the leaf level to reduce the cost of structural modifications during updates. Third, it adopts a flattened design to track on-disk data more precisely while minimizing memory overhead. Finally, LUCID incorporates a lightweight, optimistic locking mechanism tailored to the needs of larger-than-memory systems, enabling efficient concurrent access.**

***Index Terms*—Learned indexes, Larger-than-memory data, Concurrency control**

I. Introduction

Modern data management systems face two critical challenges: rapidly growing data volumes and complex, concurrent workloads. Applications like web services and IoT generate massive datasets that routinely exceed memory capacity [1, 2]. At the same time, these systems must efficiently handle diverse and concurrent operations, including high-throughput insertions and simultaneous queries from multiple clients or devices [3, 4]. Learned indexes have emerged as a powerful approach to improving data access across in-memory [4–7] and on-disk settings [8, 9]. However, deploying learned indexes in larger-than-memory environments with dynamic and concurrent workloads remains a significant challenge.

To effectively operate under these constraints, learned indexes need to simultaneously achieve memory efficiency, high update/query performance, and robust concurrency control. Although prior efforts have individually addressed subsets of these requirements, none satisfy all three simultaneously. Most learned indexes face trade-offs among these critical aspects:

✉ Corresponding author.

1) *Insertion performance and memory overhead*: Methods like ALEX [5] and LIPP [7] allocate gaps in nodes for insertions, but risk data shifts that impair model accuracy or increase node complexity, reducing performance. FITing-Tree [6] and XIndex [3] use fixed-size buffers for each node, balancing insertion and retraining cost, yet struggle with frequent structural modification operations (SMOs). Approaches like FINEdex [4] allow flexible insertions but incur substantial memory overhead due to per-record buffers.
2) *Concurrency management and memory consumption*: Techniques employing fine-grained locking—such as those in FINEdex and XIndex [3, 4], which rely on buffer-based insertions—minimize thread contention at the expense of considerable memory overhead. In contrast, coarser-grained locking solutions, like those employed by Treeline [10] and ALEX+ [11], reduce memory usage but cause increased contention due to in-place insertions within leaf nodes [12] and the large node sizes typical in learned indexes [9].
3) *Metadata overhead for disk-resident data*: Efficient disk management necessitates tracking metadata in memory at fine granularity, which can significantly impact memory consumption [2, 10].

To overcome these limitations, we propose **LUCID**, an **U**pdatable and **C**oncurrent **L**earned **I**ndex designed specifically for managing larger-than-memory **D**atasets under dynamic and concurrent workloads. LUCID's design is driven by the following core innovations:

- **Model-guided buffering**: Instead of fixed or uniformly allocated buffers, LUCID dynamically adjusts buffer sizes based on local data distributions learned by models, and optimizes buffer operations using model predictions, effectively balancing insertion performance and memory usage.
- **Overflow chaining at leaf level**: LUCID employs additional learned models to handle overflow insertions at leaf nodes, thereby isolating costly structural modifications from upper-level nodes and significantly reducing retraining overhead.
- **Balanced and decoupled locking mechanisms:** Balancing memory usage and concurrency performance, LUCID employs locking strategies at an intermediate granularity. Moreover, it decouples locking between model training data and testing data (i.e., new insertions), reducing interference and further mitigating contention, thereby enhancing parallelism.
- **A lightweight flattened structure (FS)**: To efficiently track disk-resident records, LUCID introduces a memory-efficient

metadata layout that dramatically reduces memory overhead without sacrificing fine-grained access performance.

In summary, our **contributions** include: **(1)** We propose LUCID, a novel learned index for larger-than-memory data with dynamic and concurrent workloads. **(2)** We introduce model-guided buffering and leaf-level overflow chaining to efficiently support dynamic data insertions. **(3)** We design an optimized concurrency control mechanism that balances locking granularity and memory efficiency. **(4)** We develop a lightweight flattened structure that gracefully manages out-of-memory records with minimal overhead. **(5)** We conduct extensive experiments demonstrating LUCID's superiority over baselines across diverse, realistic settings.

Outline. We present preliminaries and LUCID overview in §II, followed by the details of LUCID in §III. We then introduce update handling and concurrency in §IV. §V describes query processing. Experiments are presented in §VI, and we discuss related work in §VII. §VIII concludes the paper.

II. Preliminaries and Overview

We first revisit learned indexes and existing approaches to managing larger-than-memory data (§II-A), then outline the key principles that motivate LUCID's design (§II-B), and finally present LUCID overview (§II-C).

A. Preliminaries

1) Learned indexes. Given a sorted array of keys $a = [k_1, \cdots, k_i]$, learned indexes employ machine learning (ML) models to capture the data distribution of keys and predict the position for a given key [13]. Typically, a learned index adopts a hierarchical structure—comprising internal and leaf nodes—akin to the B+tree [14]. Each node maintains an ML model; internal nodes guide traversals through the hierarchy to reach leaf nodes that store the actual data [11, 15]. Upon reaching a leaf node, the model predicts the position of the queried key. Inaccurate predictions are refined by local search. We call the data used to train a node's model the *training data*, and newly inserted data the *testing data*.

Although advanced learned indexes have been proposed for dynamic updates [5, 7, 16, 17] and concurrency [3, 4], running them efficiently on datasets that exceed memory capacity remains a challenge. Below, we briefly review two crucial aspects: dynamic updates and concurrency control.

Dynamic Updates in Learned Indexes. As depicted in Fig.1, existing dynamic solutions typically rely on preallocated space—such as gapped arrays or buffers—and logarithmic merging. For example, ALEX [5] uses sparse nodes (Fig.1a-❶), which may cause costly data shifts on insertions [7], degrading model accuracy. LIPP, DILI and SALI [7, 18, 19] mitigate shifts via overflow nodes (Fig.1a-❷), linking keys with identical predicted positions through new nodes. However, this may suffer from long traversal paths and poor lookup performance [12]. LOFT [12] applies error-bounded insertions (i.e., placing records in nearby empty slots without shifting existing keys), but records may no longer be ordered. XIndex and FITing-Tree [3, 6] use fixed-size buffers per leaf (Fig.1b-❶); small buffers trigger frequent restructuring, while large

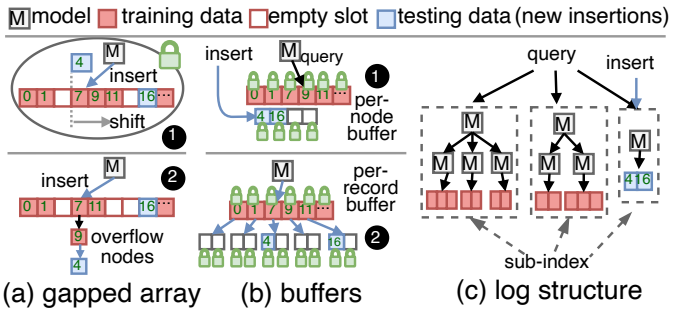

Fig. 1. Learned indexes for insertion and concurrency control

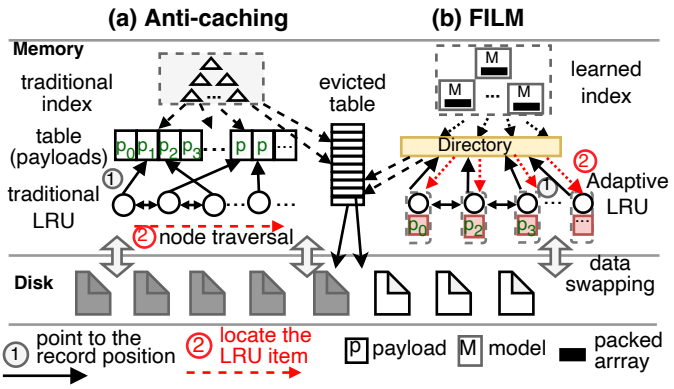

Fig. 2. Anti-caching [20] and FILM [2]

ones reduce key search performance. FINEdex [4] uses finer-grained buffers for each record (Fig. 1b-❷) for insertions but increases memory usage. PGM-index [17] builds sub-indexes (Fig.1c) for different subsets of keys to enhance efficiency, but querying requires searching across all sub-indexes [11].

These approaches either interleave training and testing data within a unified structure, or physically separate them without fully leveraging model-guided optimizations for new keys.

Concurrency Control in Learned Indexes. Existing learned indexes typically support concurrency control using optimistic locking and version control. For example, XIndex [3] and FINEdex [4] employ fine-grained per-record locks (Fig. 1b-❶❷) to minimize read/write conflicts, incurring significant memory overhead. ALEX+ and APEX [11, 16] utilize coarse-grained per-node locks (Fig. 1a-❶) to balance the lock acquisition cost and performance; however, its large node size can potentially lead to increased contention [9]. Crucially, these concurrency control methods do not distinguish between training data (already fitted by the model) and testing data (newly inserted). As a result, newly inserted data (i.e., testing data) can unnecessarily conflict with reads and writes on training data, degrading the overall performance.

2) Larger-than-memory data management. **Anti-caching** introduces a new architecture for main-memory DBMSs by expanding system capacity with disk and completely separating data across different storage [20]. To improve performance, it retains essential structures in memory, including indexes, an LRU (Least Recently Used) chain, and an evicted table (i.e., metadata for disk data), as shown in Fig. 2(a). Unlike normal

caching, anti-caching never retains copies of records. When memory is full, cold records are identified by LRU and evicted to disk. An entry (*tombstone*) is added to the evicted table to track each evicted record, and the index is updated to point to this entry instead of the original record location. Nonetheless, anti-caching faces limitations. 1) Traditional indexes, with anti-caching adopted, must reside in memory to ensure low-latency queries, consuming a large portion of memory [1, 21]. 2) Cold-data identification through LRU chain traversal incurs about 25%-35% overhead of total query time [2, 22, 23].

FILM [2] builds on the anti-caching strategy and proposes a learned index for larger-than-memory databases. As presented in Fig. 2(b), FILM constructs a hierarchy of learned models to index data stored across memory and disk. The learned models of leaf nodes are trained on the keys of data with the PGM method [17, 24], partitioning the key space into sub-ranges. The first key in the sub-range serves as the key for the model. Non-leaf nodes are constructed recursively, fitting models over child node keys until the root contains a single model.

To avoid costly retraining of learned models caused by frequent location changes during data swapping, FILM employs an in-memory directory to map keys to their locations either in memory or on disk. For cold data identification, it integrates an adaptive LRU that piggybacks the high-cost LRU maintenance onto the high-performance learned model lookup.

However, FILM is tailored for single-threaded access and append-only, sorted insertions. It uses a single buffer for the entire tree to absorb arbitrary inserts, which reduces memory usage but performs poorly under heavy arbitrary insertions due to costly whole-tree rebuilding when the buffer fills up. Moreover, directly applying existing solutions for dynamic updates to FILM results in frequent SMOs at the leaf level [7], potentially causing costly propagations up to the root. This degrades performance—especially because FILM, like PGM-Index [17], uses packed arrays at internal levels without reserved space for updates to minimize storage overhead.

Our goal is to design a learned index that caters to larger-than-memory data with frequent, arbitrary inserts and concurrency, while further improving memory efficiency.

B. Key Principles

Based on the above analysis, we identify four guiding principles for the design of LUCID.

P1. **Using memory wisely and frugally**. Managing larger-than-memory data inherently requires efficient access to data residing both in memory and on disk. Due to the significant performance gap between memory and disk access speeds [25], it is essential to store frequently accessed (hot) data and critical structures, such as indexes and metadata for disk-resident data, in memory. Therefore, a practical index must use the limited memory judiciously, keeping its structures compact, thereby maximizing the amount of data held in memory and ensuring high query and update performance.

P2. **Minimizing training-testing interference**. As described in §II-A1, training data are used to build the models, while newly inserted keys are treated as testing data. Enforcing a unified structure for both training and testing data increases unnecessary read/write contention during concurrent operations. Further, a unified structure may inadvertently alter the positions of training data when handling insertions, degrading model accuracy and impairing model-guided operations. Therefore, LUCID should decouple the handling of training and testing data to mitigate interference.

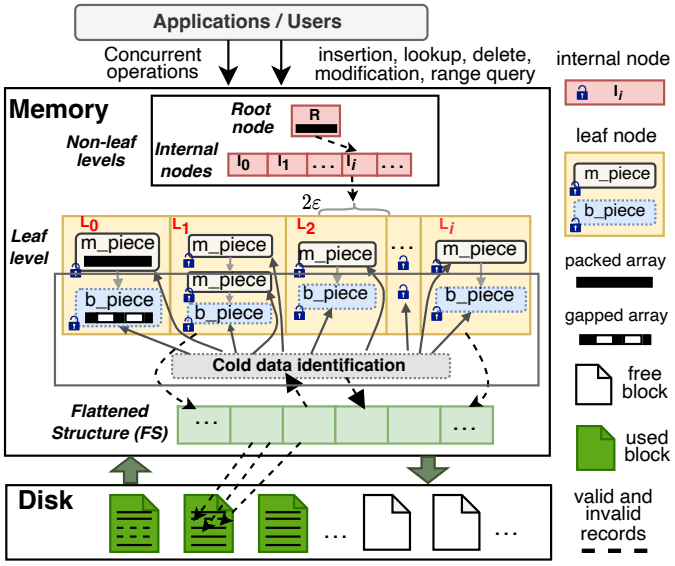

Fig. 3. LUCID: An Overview

P3. **Unified model-guided Optimization**. Models learned on training data capture key data patterns. Using these patterns during insertion and query processing can significantly optimize operations [26]. However, simply decoupled architectures (addressing **P2**) that separate training and testing data, often confine model-guided optimizations to training data. This narrow focus overlooks optimization opportunities for newly inserted testing data. LUCID aims to combine the benefits of decoupling with model-guided optimizations for both training and testing data, thereby maximizing overall efficiency.

P4. **Localizing SMOs and mitigating cascading effects**. SMOs can be heavyweight and predominantly occur at the leaf level, where most data insertions take place [7]. Cascading SMOs upwards necessitates additional measures at higher levels, such as preallocated space and node splits or expansions, which consume memory and increase complexity. To minimize storage and maintenance overhead, LUCID should confine frequent SMOs and prevent their propagation beyond leaf level.

Drawing on these principles, we propose LUCID, built atop FILM's anti-caching design [2, 20], but extended with novel mechanisms for efficient updates, concurrency, and memory management in larger-than-memory settings.

C. Overview

Fig. 3 gives an overview of LUCID, a tree-structured index. Each node hosts learned models for prediction. Leaf node models are fitted on data sub-ranges partitioned according to data distribution, while non-leaf models are trained on the first keys of their child nodes from bottom up, to predict the set

of nodes to be accessed in the next level. Like FILM, LUCID employs adaptive LRU for efficient cold data identification.

In the following, we highlight the core innovations of LUCID, connected to the principles in §II-B:

1) **Model-guided buffering (P2, P3):** Each leaf node contains a model component and a buffer component. The model component houses at least one model *piece* (*m_piece*), which is trained during bulk loading, containing the training data for that model. The buffer component features a single buffer *piece* (*b_piece*) pre-allocated based on the data distribution of *m_piece*, to accommodate newly inserted keys (testing data), mitigating the interference between training and testing data (**P3**). New keys are inserted to *b_piece* using the prediction from *m_piece*, followed by model-guided lookup to boost performance (**P2**) (§III-A and §IV-A).
2) **Overflow chaining at the leaf level (P4):** Once the *b_piece* is full, LUCID retrains the leaf node by combining the data in *m_piece* and *b_piece*. The retraining may result in one or more *m_piece*s, depending on the data distribution. When several *m_piece*s are generated, instead of recursively propagating these changes to the upper levels—which would trigger costly model retraining in non-level nodes—LUCID links the resulting *m_piece*s into an overflow chain within the leaf node. This design localizes expensive SMOs to the leaf level, isolating them from non-leaf levels (§IV-A2).
3) **A lightweight flattened structure (FS) (P1):** Keeping fine-grained metadata in memory for on-disk data is crucial for reducing disk I/O and improving query performance [10, 20]. To retain this benefit while minimizing memory usage, LUCID develops a flattened structure (FS) that consolidates tracking metadata into a single, type-unified array with bit-level encoding. This compact design simplifies the data structure and reduces memory overhead (§III-C).
4) **Piece-level locking with training-testing decoupling (P1, P2):** To balance memory efficiency and contention reduction, LUCID employs per-piece locking rather than the coarse-grained per-node and fine-grained per-record locking (**P1**). Additionally, it enables decoupled locking for training and testing data, as insertions are confined to *b_piece*, further mitigating read/write conflicts during concurrent operations (**P2**). We detail concurrency control in §IV-C.

LUCID thus offers a learned index tailored for larger-than-memory data, enabling dynamic updates and concurrent operations, while maintaining low memory overhead.

III. The LUCID Index

This section details LUCID's components: node structure, cold-data eviction, and the FS for tracking on-disk data.

A. Node Structure

LUCID features two types of nodes: internal nodes and leaf nodes. Each node maintains a linear model *M* fitted on a sorted array $a = [k_1, \cdots, k_{|a|}]$. The model is represented by a triple $\{key, sl, ipt\}$. Specifically: *sl* and *ipt* represent the *slope* and *intercept* of *M*, while *key* is the first key in its sub-range (for leaf nodes) or the first key among its child nodes (for internal nodes). The model, partitioned following the same approach

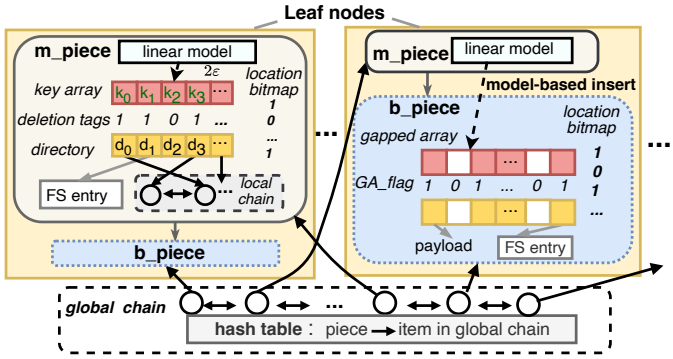

Fig. 4. Leaf nodes in LUCID

as in PGM-Index [17, 24] satisfies ε-*approximation* [11, 15], $|M(k_i) - i| \leq \varepsilon$, where k_i represents a queried key, $M(k_i)$ is the position of k_i predicted by M, and the error bound ε denotes the maximum prediction error of *M*, i.e., the upper bound of distance between $M(k_i)$ and i.

Internal nodes are responsible for predicting the corresponding nodes to be accessed at the next level. The prediction is further refined through local searching to determine the exact position. To reduce storage overhead and simplify node management, LUCID adopts a packed array structure (without free space) for internal nodes, where each entry points to a corresponding child node, similar to strategies in [2, 17].

Leaf nodes partition the key space into non-overlapping sub-ranges, with each consisting of two components (Fig. 4)—*m_piece* (assuming a single *m_piece* for now for ease of exposition) and *b_piece*—specifically tailored to support dynamic and concurrent workloads in larger-than-memory databases. At a high level, the *m_piece* comprises a learned model fitted on training data. The *b_piece* serves as a buffer for the *m_piece*, accommodating new keys (i.e., testing data) with predictions from the same model of *m_piece*. We next detail the design of *m_piece* and *b_piece*.

1) m_piece. Fig. 4 illustrates the design of *m_piece*, including: a) A *linear model* that predicts a key's position $p\hat{o}s$ in the corresponding leaf. b) A *packed key array* storing this *m_piece*'s keys. During lookups, LUCID retrieves all keys within 2ε of $p\hat{o}s$ for the exact matching. c) *Deletion tags* indicating whether each key is valid or deleted. This avoids key movement, thereby preserving the ε-*approximation* guarantee despite deletions. d) A *local chain*, adapted from FILM [2], that stores the payloads of in-memory keys in LRU order and supports cold data eviction, as described in §III-B. e) A *directory*, aligned with the key array, that contains pointers dynamically tracking the physical location of each key's payload, a memory address (if in memory) or a disk address (if on disk). f) A *location bitmap* that works in conjunction with the directory to resolve each record's actual location: "1" for memory (the local chain) and "0" for disk referencing addresses in the flattened structure (§III-C). g) A pointer to the leaf's *b_piece*, as discussed in §III-A2.

Compared to FILM [2], LUCID's *m_piece* introduces sev-

eral key differences to address the unique requirements of handling dynamic updates. First, the *m_piece* has a buffering structure to accommodate newly inserted keys that fall within its key range. Second, LUCID extends the model to handle both *m_piece* and *b_piece* keys, maximizing model-guided optimization for training and testing data. Finally, it features lightweight deletion tags to efficiently support deletions.

2) b_piece. As illustrated in Fig. 4, *b_piece* resembles *m_piece* but differs in how it manages new keys:

a) The *b_piece* serves as the buffer for *m_piece* and is empty at initialization. LUCID assumes that newly inserted keys (i.e., testing data) hitting a particular leaf exhibit a similar data distribution to the training data in *m_piece*. Moreover, LUCID can adapt to distribution skew, as described in §IV-C3.

b) In *b_piece*, the key array and directory are structured as two aligned gapped arrays (GAs, i.e., arrays that leave empty slots between records). We allocate a fraction *buffer_ratio* of the leaf node's space (whose size is determined by data distribution) to *b_piece*, and remaining space to *m_piece*. We analyze the impact of different *buffer_ratio*s in §VI-E2.

c) Each *b_piece* maintains a bit array (GA_flag) to track whether a slot in the GA is occupied by a record or is a gap.

d) Rather than maintaining deletion tags to indicate whether a key has been deleted, *b_piece* performs direct deletions. Since key position changes in *b_piece* do not affect model accuracy, this also helps free up buffer space for future insertions.

e) Instead of using the local chain to store the payloads of in-memory data (maintaining the LRU orders), *b_piece* directly places payloads in the aligned directory for three reasons: 1) Inserting a new record into a non-gap position causes data shifting, which frequently changes record offsets. This incurs additional latency for insertion, as the original design of local chain relies on offset to point to key's position in key array [2]. 2) It is reasonable to assume that all the newly inserted data in *b_piece* are hot as per [20, 27]. 3) The aligned "payload array" (i.e., directory) also reduces the memory consumption caused by the bidirectional pointers in the local chain.

Upon insertion, *m_piece*'s model predicts the position $p\hat{o}s$ for the key. If a gap exists, the key is placed there; otherwise, minimal shifting is performed to create a gap near $p\hat{o}s$. This design differs from ALEX [5], which uses a single gapped array for all keys and can provoke "unbounded" shifts [7]. By isolating the testing data in *b_piece*, LUCID confines shifts to a much smaller region, preserving the performance of *m_piece*.

The *m_piece* and *b_piece* together enable efficient, model-guided insertions. Each *b_piece*'s capacity is determined based on the training data distribution, with larger *m_piece*s anticipating higher insertion volumes and requiring larger *b_piece*s. Moreover, the same learned model from *m_piece* guides insertions in *b_piece* (see details in §IV-A).

B. Cold Data Eviction

Whenever memory is exhausted, LUCID identifies cold data using an adaptive LRU scheme [2], and then evicts them to disk blocks. We adopt this strategy for its low overhead (as described in §II-A2). However, its application in LUCID involves significant modifications as discussed below.

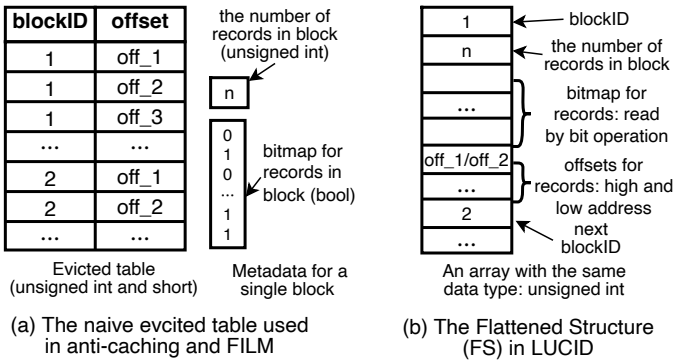

Fig. 5. Evicted table [2, 20] vs. LUCID's flattened structure (FS)

Fig. 4 shows how LUCID maintains a *global chain* to track the access order of *m_piece*s and *b_piece*s—unlike FILM [2], which tracks entire leaf nodes. Since *m_piece* and *b_piece* within the same leaf may exhibit different access patterns, LUCID's piece-level global chain offers finer-grained control. Meanwhile, each *m_piece* has a *local chain* to preserve the LRU order of its in-memory records, while newly inserted records in *b_piece* are treated as hot [20, 27].

For queries accessing in-memory data, LUCID promotes the accessed *m_piece* or *b_piece* to the head of the global chain. If it is an *m_piece*, the specific key in the local chain is also promoted. For disk-resident data, once the requested record is fetched, it is merged into the local chain (for *m_piece*) or the gapped array (for *b_piece*), as described in §III-C.

During eviction, LUCID locates the least recently used *piece* from the global chain's tail. For *m_piece*, it consults the local chain to pick cold data; for *b_piece*, it examines the gapped array. Cold records are then evicted to disk blocks, following [2].

We next describe how LUCID manages these evicted records to facilitate query performance and reduce overhead.

C. The Flattened Structure (FS)

In a larger-than-memory setting, LUCID tracks each evicted record in memory with a pair $\langle blockID, offset \rangle$ to facilitate queries. For each disk block, metadata is stored, including the number of valid records and a bitmap signifying invalid offsets ("0" means that the record has been merged back to memory).

When a query requests records from a disk block, LUCID reads the entire block into memory. To improve efficiency, only the requested records are merged back into the leaf nodes. The bitmap is then updated to mark these offsets as invalid ("0" bits), creating "holes" in the block. To manage these holes, LUCID employs a lazy merge approach: if the fraction of invalid offsets in a block exceeds a user-defined threshold, LUCID merges all valid records from that block into memory in a single pass, reducing unnecessary incremental moves. We empirically study the merge threshold in §VI-C3.

As depicted in Fig. 5(a), prior studies [2, 20] maintain an in-memory evicted table to track on-disk data, causing memory waste due to duplicates of the same *blockID*. Additionally, the metadata for each block is stored in separate structures. LUCID addresses these inefficiencies with a *flattened structure*

(FS), as shown in Fig. 5(b), which consolidates all block-tracking information into a single array of a uniform data type (unsigned integers in our implementation), simplifying the data structure and minimizing memory overhead.

To clearly illustrate the advantages of FS and see how it works, let us consider an example where the block size is 64 KB and each record is 128 bytes. In a naive evicted table setup, blockIDs and record counts per block are stored as 4-byte unsigned integers (32-bit uint), offsets as 2-byte short ints, and a bitmap as an array of booleans. In contrast, the FS strategy optimizes storage by using uint as the base data type for all elements: it encodes bits of the bitmap within the uint itself and incorporates the offset as either the upper or lower 2 bytes (16 bits) of the uint. For instance, when LUCID evicts a record to block i (a uint) at offset off_1 (2-byte uint, sufficient to encode the positions of records within a block), FS updates the upper 2 bytes of the corresponding entry to store the offset, flips the relevant bit in FS via a bitwise operation, and increments the block's record count by one. Comparatively, the naive method requires 3,140 bytes to store tracking information for a single block, whereas the FS method significantly reduces this requirement to 1,096 bytes, achieving a storage saving of approximately 65%.

The FS enables LUCID to track and access disk records with less memory overhead and simpler metadata handling.

IV. Handling Updates

This section details how LUCID handles insertions and deletions (§IV-A, §IV-B), followed by its concurrency (§IV-C).

A. Model-guided Insertions

Given a record to insert, LUCID first traverses from the root to the correct leaf node, using each internal node's model to guide the descent. Upon reaching the leaf, LUCID employs the leaf's *m_piece* model to predict the insertion position for the new record. The subsequent procedure depends on whether the leaf's *b_piece* is non-full or full, as discussed below.

1) Insertion in a non-full b_piece. As shown in Fig. 6(a), if *b_piece* has available gaps (that is, it is not full), LUCID inserts the new record with key k as follows:

a) Use the leaf's *m_piece* model to predict an insertion position, $p\hat{o}s = (k \times sl + ipt) \times b_f$, where sl and ipt are model parameters, and b_f denotes the buffer ratio (§III-A2).

b) If the predicted position $p\hat{o}s$ violates the sorted order – specifically, if the position after $p\hat{o}s$ contains keys smaller than k – a short local search is performed to refine the position. In LUCID, gaps are filled with the rightmost non-gap keys, facilitating verification and accelerating queries.

c) If the position is a gap, insert directly; otherwise, shift minimal records toward the nearest gap to create space.

This design differs from ALEX [5], which may trigger "unbounded" shifts in a single large array [7]. By isolating new data in *b_piece*, LUCID confines shifts to a smaller region. The *m_piece* (training data) remains unaffected, preserving its fast search performance. Additionally, unlike LOFT [12], LUCID maintains sorted order for efficient range queries.

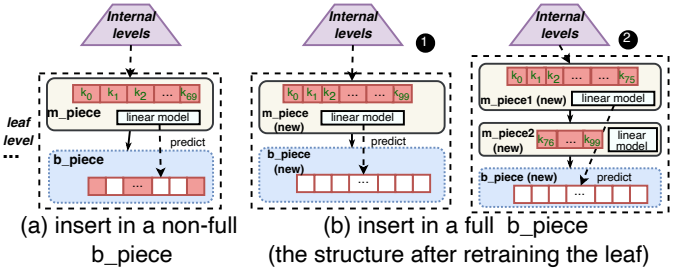

Fig. 6. Insertion in LUCID

Algorithm 1 Insertion in a full *b_piece* and the overflow chain

Require: the leaf node (with *m_piece* and a full *b_piece*) where k is inserted
Ensure: the retrained leaf node

```
if b_piece is full (no available gaps) then
    Combine data in both m_piece and b_piece
    Retrain leaf node and allocate new b_piece with free space
    The retraining may generate one or more new m_pieces
    if the number of m_pieces = 1 then
        Replace old m_piece with the new one
    else
        Link all m_pieces with key order in a chain      % overflow chain
if SMO threshold reached then
    Merge m_piece chains at leaf level
    Rebuild internal nodes as necessary
```

2) Insertion in a full b_piece. When the *b_piece* in the leaf node becomes full (i.e., no available gaps) after insertions, we retrain the leaf by merging the data from *m_piece* and *b_piece*, following PGM method [17] to satisfy ε-*approximation*. Retraining may result in one or more *m_piece*s (Fig. 6(b)), depending on the data distribution. If only one *m_piece* is fitted, it will simply replace the old one. The second case requires more consideration where several *m_piece*s are generated, suggesting a distribution skew. A seemingly straightforward solution is to insert each of the new *m_piece*s at the leaf level so that each leaf node contains one *m_piece*. However, the changes are then propagated recursively to the upper levels, causing array reallocation (due to the packed array design) and model retraining in internal nodes.

Overflow chain at the leaf level. To avoid such expensive operations, we replace the old *m_piece* with a chain of the newly trained *m_piece*s in the same leaf node (Fig 6(b)-❷). These *m_piece*s are linked by key order and share a common *b_piece*. Subsequent insertions will be placed in the shared *b_piece* based on the prediction of the largest *m_piece* in the chain, which has proven effective in our experiments.

To bound chain traversal depth, LUCID designs a threshold to periodically do structural modification operations (SMOs), which merge the leaf level (i.e. unlink all the *m_piece* chains) and rebuild the internal levels (detailed in §IV-C). The whole procedure is outlined in Algorithm 1.

3) Cost Analysis. The overall insertion time is $O(\log(\frac{n}{2\varepsilon}) + \log b)$, covering tree traversal and local searching in *b_piece* of size b, where $\frac{n}{2\varepsilon}$ is the worst-case number of leaves [28]. As empirically shown in §VI-B, local searches are usually close to ε, which is much smaller than $\log b$.

Other updates. Modifying non-key attributes involves lo-

cating the record and changing the payload in place if it is in memory. If on disk, the new payload is inserted into local chain (*m_piece*) or gapped array (*b_piece*), and the associated metadata is updated to mark the on-disk version as invalid (§IV-B).

B. Handling Deletion

The data in LUCID reside in either *m_piece* or *b_piece*, we handle deletions differently for each:

- **Deletion in *m_piece*.** LUCID uses a binary token (0/1) to mark whether each key is deleted (deletion tags in §III-A1). This avoids physically shifting keys, which could violate ε-approximation. Invalid entries are reclaimed during leaf retraining (as described in §IV-A2).
- **Deletion in *b_piece*.** Since shifting *b_piece* does not affect the model, keys can be deleted directly in place, freeing up buffer space for future insertions (as evaluated in §VI-B2).

The above process efficiently handles in-memory data deletions. However, in larger-than-memory settings where data span both memory and disk, LUCID must determine the location of the data to be deleted. For on-disk deletions, LUCID either marks the token in *m_piece* or removes the key from the *b_piece*, while also updating metadata. This includes adjusting the count of valid records of the affected block and updating the bitmap to mark invalid (i.e., unusable) offsets. These invalid spaces are reclaimed during block merges (§III-C). This approach minimizes disk I/O and improves block reuse.

Deletion incurs a similar cost to insertion (§IV-A3). We omit the analysis for brevity.

C. Concurrency Control

This section discusses concurrency control in LUCID. Conflicts occur when multiple threads concurrently execute write/write or read/write operations on the same leaf node. We demonstrate the adaptation of optimistic locking and version control techniques to LUCID, which have been successfully applied in FINEdex and XIndex [3, 4]. However, their application in LUCID involves significant modifications..

1) Write/Write coordination. To conserve memory, LUCID utilizes *per-piece* (instead of *per-record*) locks and version numbers. By separating the training data in *m_piece*s and the testing (newly inserted) data in *b_piece*, the impact of concurrent insertions is localized to *b_piece* only. During insertion, the target leaf is first identified, and the lock on its *b_piece* is then acquired to prevent interleaving with concurrent writers. After insertion, the version of *b_piece* is increased and the lock is released. Concurrent updates of existing records resemble inserts and involve acquiring the per-piece lock of the affected *m_piece* or *b_piece* and increasing the version.

2) Read/Write coordination. To reach the target leaf node containing the request data, the reader thread traverses from the root to the leaf level without acquiring any locks in internal levels (i.e., reading the i_piece is lock-free). Then the reader tries to read the requested key from the key array in the *m_piece* or *b_piece* and snapshots the version before fetching the payload from memory or disk. After the payload is fetched, the reader verifies the version number to detect inconsistent or stale data and ensure no lock is held on the target *m_piece* or *b_piece* (i.e, no concurrent writes). If verification fails, the reader retries the process until a successful read is carried out.

Concurrent deletions follow a process similar to insertions: the corresponding *piece* is first locked to avoid interference from concurrent writers. Readers are not blocked.

3) Concurrent retraining. As outlined in §IV-A2, a full *b_piece* will trigger concurrent retraining in LUCID to reduce blocking of other operations. This is highly beneficial to make LUCID adjust to new data insertions and the changing distributions—such as data inserted to different sub-ranges with varying frequencies—especially in multi-core environments. To improve concurrency, LUCID conducts two types of retraining at different frequencies, including single-leaf retraining and internal-level retraining.

Single-leaf retraining. As a *b_piece* becomes full after insertion, single-leaf retraining will retrain the models based on all the data in that leaf, including those in *m_piece*s and *b_piece*, and create more space by allocating a larger *b_piece* for future insertions. To avoid blocking lookups, leaf retraining is performed out-of-place by allocating a new leaf node.

Specifically, LUCID first locks the target leaf to ensure only one thread can conduct retraining on it. During retraining, a temp buffer is allocated to absorb the inserted records to ensure data consistency and zero data loss. Once the retraining is complete, LUCID releases the lock and uses the newly retrained leaf to replace the old leaf. If the temp buffer contains any records, LUCID inserts them into the *b_piece* of the new leaf. Finally, the old leaf will be deleted from memory.

Internal-level retraining. Inevitably, some leaf nodes may develop a long *m_piece* chain due to leaf retraining with more insertions, as shown in Fig. 6(b)-❷. LUCID periodically executes internal-level retraining to avoid the traversal of long *m_piece* chains. This process first unlinks the *m_piece* chain by allocating each *m_piece* to a newly created leaf node. Then, the internal levels are recursively rebuilt from bottom to up, to guarantee the prediction accuracy of the internal levels.

As internal-level retraining is infrequent and typically completes in milliseconds, LUCID conducts internal-level retraining in an exclusive mode that blocks other concurrent operations. Nonetheless, non-blocking internal-level retraining remains an interesting future direction. For example, retraining internal levels could proceed non-in-situ by creating a new tree without blocking lookups on the old tree, while simultaneously acquiring the lock of the root node and allocating a global temp buffer to handle all the insertions during retraining.

In summary, through the two types of retraining, LUCID improves the concurrency performance and achieves dynamic adjustment to the distribution skew caused by data insertions.

V. Query Processing and Cost Analysis

A. Point and Range Queries

1) Point query. To locate a record with a given key *k*, LUCID performs three main steps:

a) **Tree traversal.** Starting from the root, LUCID uses the model at each internal node to predict which child node to access. Due to the $\varepsilon\text{-}approximation$ guarantee (§III-A), any

prediction error is corrected with a local search at each level. This process continues until reaching the target leaf node L_i.

b) **Locate *k* within the leaf.** If L_i contains a chain of *m_piece*s, LUCID identifies the relevant *m_piece* based on k's value, as *m_piece*s are sorted. It predicts k's position using the linear model ($\hat{pos} = k \times sl + ipt$) and then searches within $\pm\varepsilon$ of $\hat{pos}$. If not found in *m_piece*, LUCID consults the *b_piece*, using either the same model or the model from the largest *m_piece* in the chain to guide the search.

c) **Retrieve the record.** If k is in memory, LUCID accesses it directly from the local chain, where it is also moved to the head (for *m_piece*), or from gapped array (for *b_piece*). The global chain is updated to promote the accessed $piece$. If k resides on disk, LUCID retrieves the corresponding $\langle$blockID, offset$\rangle$ from the aligned directory, fetches the disk block, and updates LRU information.

2) Range query. For a range query starting from key k, LUCID first locates k using the same traversal procedure. It then performs a leaf-to-leaf scan, retrieving all subsequent keys until the range is covered. Both *m_piece*s and *b_piece* in each leaf are checked. When disk blocks are needed, LUCID identifies them in a preprocessing step and performs batched I/O to reduce overhead. LRU is updated to reflect accesses.

B. Cost Analysis

For in-memory access, the point query time is $O(\log \frac{n}{2\varepsilon} + \varepsilon + \log b)$, where $\log(\frac{n}{2\varepsilon})$ corresponds to tree traversal (with n keys and error bound ε), ε accounts for local searches in *m_piece*, and $\log b$ represents searching *b_piece*. Model-guided placement often reduces *b_piece* search costs, as empirically studied in §VI-B. For range queries, the same initial overhead applies, plus $O(n_k)$ to scan n_k keys across the relevant leaves.

When records reside on disk, each access may involve an $O(p)$ disk operation to fetch the relevant blocks (or batched operations for multiple blocks), where p is the number of fetched blocks. Overall, LUCID's query performance is dominated by tree traversal and local searches for in-memory data, and by I/O for on-disk data.

VI. Experiments

A. Experimental Setup

1) Baselines. We compare LUCID against nine state-of-the-art traditional and learned indexes that are applicable to our target setting: a) **B+tree** [14] uses preallocated space at the end of each leaf node. b) **ALEX** [5] employs gapped arrays that distribute extra space between keys in the array. c) **FITing-Tree** [6] (FIT) maintains per-node buffers at leaf level. d) **FINEdex** [4] (FINE) uses fine-grained per-record buffers and supports concurrency at the record level. e) **FILM** [2] adopts a single global buffer for the entire tree. f) **XIndex** [3] applies per-node buffers and per-record locks. g) **AULID** [9] is a practical on-disk learned index. h) **Treeline** [10] designs a record-level caching solution for larger-than-memory datasets. i) **HybridPGM** [29] (HPGM) is developed based on the "SGACRU" guidelines proposed in [29], which converts in-memory learned indexes into efficient disk-based versions.

Among these, AULID, Treeline, and HPGM were originally designed for larger-than-memory data. For the others, we adapt each method for larger-than-memory settings using *anti-caching* [20], following prior work [2], to support queries that access data not in memory (see §II-A2). For concurrent settings, we compare LUCID with FINE, XIndex, HPGM, and B+tree, which support concurrency in record- or node-level either out of the box or by minor adaptation.

2) Environment and Implementation. All experiments run on an Ubuntu machine with two 8-core Intel CPUs (3.6 GHz, 64 KB L1 cache, 1024 KB L2 cache), 128 GB memory (4×32 GB), and a 557 GB disk. We use direct I/O to bypass OS caching. Except in §VI-C4, experiments are single-threaded.

We implement LUCID in C++ (g++ 10.2.1). For baselines, we use STX B+Tree [30] with optimally tuned parameters; for ALEX, FINE, FILM, XIndex, Treeline, AULID, and HPGM, we use open-source code from [31–37]; FIT with our own C++ implementation of the design from [6]. For AULID, we keep all the internal nodes in memory, and we make the same amount of memory available to all the other methods.

For cold data identification, LUCID uses adaptive LRU with local/global chains (§III-B). The other baselines do not feature local chains, so we equip them with hLRU—an LRU chain accelerated by a hash index for fast updates. Since traditional LRU or sampling-based LRU (aLRU) [20] introduces higher chain traversal overhead [2], we adopt hLRU for B+tree, ALEX, FIT and FINE for a more fair comparison.

3) Datasets. We use 14 real-world datasets from benchmarks [11, 13] plus one synthetic dataset generated with YCSB [38] using the tool of [21]. Each dataset contains at least 60M 64-bit keys. The distributions of these datasets vary in terms of how challenging it is for models to fit, from *easy* (e.g., wise, books) to *hard* (e.g., planet and fb), as per the definition of hardness from [11]. Since each dataset provides only keys, we attach a default 120 B payload [2, 20] to form a 128 B record. The impact of record size is evaluated in §VI-D1. The size of dataset YCSB is up to 128 GB to test the big-data performance. More details can be found in [39].

4) Workloads. In experiments, we issue insert/query requests with these ratios: *read-heavy* (80% queries, 20% inserts), *balanced* (50% queries, 50% inserts), and *insert-heavy* (20% queries, 80% inserts).

Query keys follow four patterns (*zipf*, *random*, *hotspot*, and *zipf+random*). For *zipf*, the keys are selected according to the zipfian distribution, where newer items are accessed more frequently [20]. For *random*, the keys are evenly and randomly sampled [17]. For hotspot, the keys are randomly sampled from a small part of the dataset [4, 13]. For *zipf+random*, the keys are drawn from a hybrid of zipfian and random distribution [5]. Range queries retrieve up to 100 consecutive keys as in [5]. We study the effect of range size (i.e., the number of keys in a range) on LUCID in §VI-D2. For deletions, we randomly select keys from the existing keys; by default, 20% of the operations are deletions, and the remaining are inserts/queries following the ratios described previously.

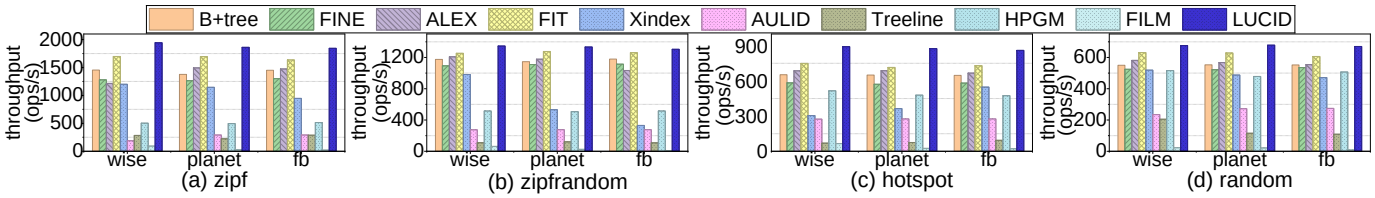

Fig. 7. LUCID vs. Baselines: insertion and point-query throughput (*read-heavy* workloads)

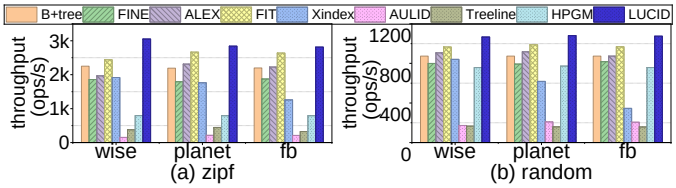

Fig. 8. Insertion and point-query throughput (*balanced* workloads)

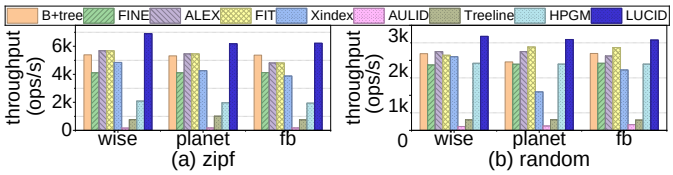

Fig. 9. Insertion and point-query throughput (*insert-heavy* workloads)

Tab. I
DISTANCE BETWEEN PREDICTED AND TRUE POSITIONS ON *wise*

Distance	read-heavy		balanced		insert-heavy	
	ALEX	LUCID	ALEX	LUCID	ALEX	LUCID
50%-tile	1	0	1	0	1	0
99%-tile	90	1	122	1	126	1
MAX	420	9	28732	8	28672	8

Experiments Outline. We organize experiments as follows, running each trial three times and reporting the average result:

1) Comparison with baselines (§VI-B), including the insertion, query, and deletion, as well as storage overhead;
2) Investigating LUCID's sensitivity on system-level parameters (§VI-C), including available memory, block size, block merge threshold, and the number of concurrent threads;
3) Evaluating the influences of data and workload characteristics (§VI-D), including record size, datasets, and range size;
4) Analyzing the impacts of model and structure parameters (§VI-E), including the ε, *buffer_ratio*, and SMO threshold.
5) Ablation study for the components of LUCID (§VI-F).

B. Comparison with Baselines

This experiment uses 4 GB real datasets by default, with the ratio of data size to available memory (Θ) set to 2:1, unless stated otherwise. We further evaluate scalability with larger datasets in §VI-B5 and examine the impact of Θ in §VI-C1.

1) Insertion and query performance. We measure the insertion and query throughput of all methods on three real datasets (*wise*, *planet*, *fb*). According to the data hardness metric in [11], these datasets exhibit varying hardness (from *easy* to *hard*); others show similar trends. Each index is initialized with $1.5 \times \Theta$ records and then serves 3 million operations.

As shown in Figs. 7-9, LUCID outperforms baselines across all insert/query ratios and workload patterns. Specifically, in *read-heavy* scenarios, it attains up to 22%–412% higher throughput than B+tree, FINE, ALEX, FIT, XIndex, AULID, Treeline, HPGM, and FILM. Under *balanced* and *insert-heavy*, LUCID demonstrates comparable gains, executing about 10%–567% more operations than baselines. FILM suffers severe degradation under frequent arbitrary insertions, as inserts trigger expensive full-tree rebuilds. As FILM is primarily designed for append-only workloads, we omit its results in *balanced* and *insert-heavy* settings.

In Fig. 11, we evaluate the performance on range queries by reporting keys accessed per second. For *balanced* workloads, LUCID almost consistently outperforms the baselines, achieving up to 9%–169% higher throughput across datasets and access patterns, with similar trends observed in other workloads. The only exception arises in random range queries, where LUCID ranks second only to HPGM. This is because HPGM stores data contiguously on disk, enabling a single read to fetch more keys within a queried range. However, under all other query patterns (e.g., *zipf*), HPGM exhibits substantially lower throughput than LUCID.

LUCID's superiority stems from the improved memory efficiency—allowing more data to be kept in memory—and from model-guided buffering, which reduces insertion and query latency. Next, we evaluate its buffering benefits.

Model-guided buffering. We record the predicted and true positions during buffer region searches in LUCID and compare them with ALEX, which mixes the training and testing data in gapped arrays. Tab. I shows the 50%-tile, 99%-tile, and MAX *distances* between predicted and true positions. The 50%-tile difference is small for both, indicating the benefits of model-guided insertion and lookup. But ALEX occasionally exhibits large errors in 99%-tile and MAX distance, e.g., 28k, as newly inserted and training data reside in one large array. In contrast, LUCID isolates new inserted data in *b_piece*, keeping training-data search unaffected. The results show that model-guided buffer achieves high performance, ensuring lookup operations satisfy the user-defined error bound (ε), even with a large-size buffer ($\gg \varepsilon$), aligning with the complexity analysis in §V-B.

2) Deletion performance. We compare LUCID with B+tree, FINE, Xindex, and HPGM, both supporting delete operations. In Fig. 12(a), we fix 20% deletions and vary the insert/query ratios. LUCID consistently outperforms baselines, achieving 10%-35% higher throughput, with notable gains in insert-heavy workloads. Fig. 12(b) further evaluates LUCID with

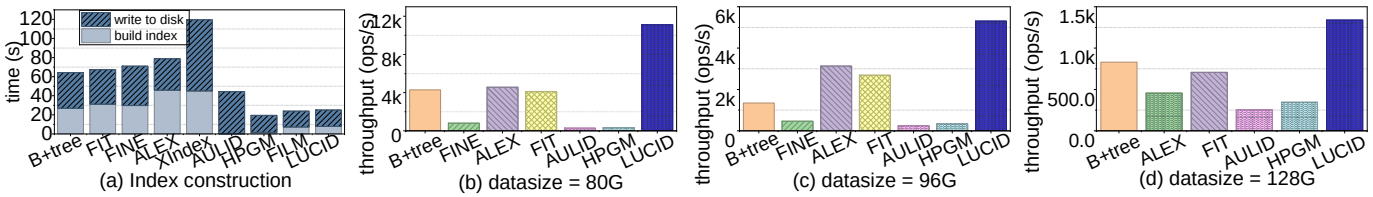

Fig. 10. LUCID vs. Baselines: (a) index construction; (b, c, d) the scalability w.r.t. larger data size

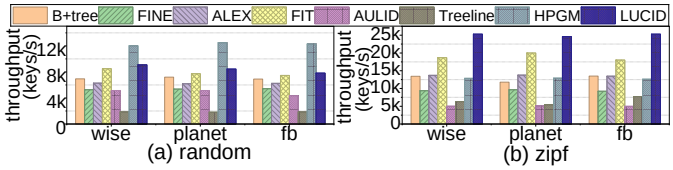

Fig. 11. Insertion and range-query throughput (*balanced* workload)

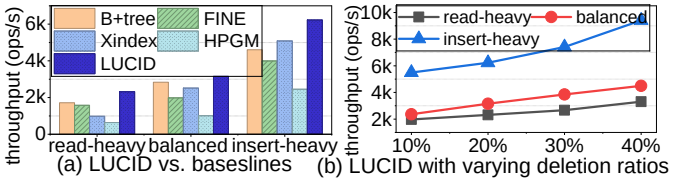

Fig. 12. Deletion performance

Tab. II
LUCID VS. BASELINES: STORAGE OVERHEAD

Used Memory(%)	B+tree	FINE	FIT	ALEX	XIndex	FILM	LUCID
data usage	43.47	39.54	50.87	47.13	30.04	68.46	63.74
LRU usage	20.38	17.30	23.85	22.09	15.0	11.39	10.02
track usage	22.46	22.90	21.64	22.06	23.95	19.70	17.50
index size	13.69	20.26	3.64	8.72	31.01	0.45	8.74

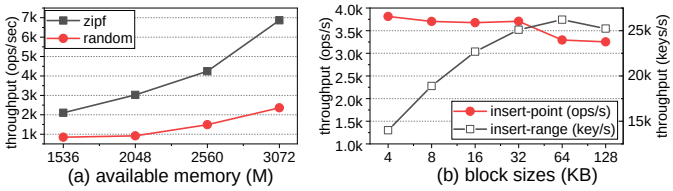

Fig. 13. Varying available memory and block sizes

varying deletion ratios (10%-40%). Throughput increases with a higher deletion ration, as deletions are primarily in-memory operations. Insert-heavy workloads benefit more substantially, as deletions free up *b_piece* space for future insertions.

3) Index construction. We also measure the build time and eviction process during index initialization. Fig. 10(a) shows results for dataset *wise*. LUCID builds its index 66%–85% faster than most baselines, primarily because they insert each loaded key into hLRU, adding overhead. HPGM employs a parallel build mechanism, leading to lower construction time. Additionally, LUCID requires 40%–81% less time to evict data due to its improved memory efficiency.

4) Storage overhead. This experiment compares memory usage after bulk-loading dataset *fb* under the same setting as §VI-B, with each method allocated 2048 MB of memory to manage larger-than-memory datasets. We break memory usage into: *data usage*: in-memory records; *LRU usage*: memory for the LRU chain and its hash table in all methods; *track usage*: directory, location bitmaps, and the metadata for on-disk data; *index size*: the learned model or tree structure, and the preallocated buffer spaces. This evaluation focuses only on methods with anti-caching architecture. Memory usage for Treeline, HPGM, and AULID is not reported. Tab. II shows LUCID incurs 20%–53% less overhead than B+tree, FINE, FIT, ALEX, and XIndex, demonstrating its superior storage efficiency. LUCID's reduced storage overhead stems from two aspects. First, its track usage is 11%-28% smaller than that of baselines, as less data are evicted to disk due to better memory efficiency. Second, LUCID applies a more lightweight FS for fine-grained tracking of evicted data. Although FILM has a smaller index size due to the single buffer, it causes long latency in frequent inserts at arbitrary positions.

5) Scalability w.r.t. large data size. Next, we allocate 64 GB of available memory and scale the YCSB dataset up to 128 GB. Fig. 10(b-d) show the throughput on a balanced *zipf* workload. Treeline and XIndex suffer from long run times, so we omit their evaluation. As data grows from 80 GB to 128 GB, all indexes slow due to increased disk I/Os. FINE struggles with per-record buffers, leading to higher memory overhead and excessive disk I/Os, so we omit its 128 GB result due to poor performance. LUCID consistently outperforms others and experiences a gentler performance drop as data size increases.

C. Sensitivity on System-level Parameters

1) Available memory Θ. We vary Θ for *wise* dataset of 4 GB under *zipf* and *random* workloads. Other datasets and workloads exhibit similar trends. Fig. 13(a) shows that as Θ increases, throughput improves because more data remain in memory and fewer disk I/Os occur. The *zipf* workload benefits more from caching hot records. Conversely, a smaller Θ leads to more frequent evictions and degraded performance.

2) Block size. Next, we vary the block size from 4 KB to 128 KB, where a larger block means more records per disk block. Fig. 13(b) reports the performance of *wise*. Similar trends are observed in others. Insert and point-query throughput is largely insensitive to block size, as each query reads only one block. In contrast, range query performance improves significantly as block size increases from 4 KB to 64 KB, due to fewer block accesses. However, beyond 64,KB, the overhead of reading larger blocks outweighs the benefits.We recommend 32–64 KB blocks and set 64 KB by default.

3) Block merge threshold. We study the impact of block merge thresholds. As discussed in §III-C, LUCID executes a

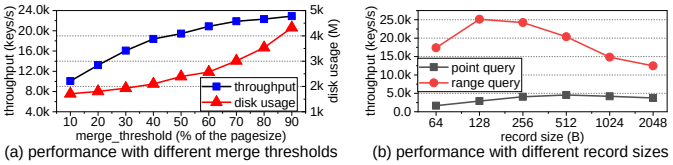

Fig. 14. Varying merge thresholds and record size

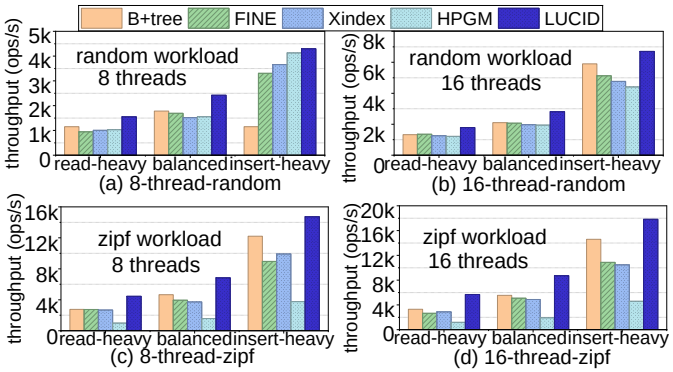

Fig. 15. LUCID vs. baseline: concurrent performance in different workloads

block merge when the percentage of "holes" (invalid records) in a block reaches the threshold. Fig. 14(a) shows performance on *balanced* workload with range queries as reads. We omit the results for point queries, as they retrieve at most one record per block, making the impact negligible. A lower merge threshold results in fewer "holes" and more frequent block merging, incurring less usage of disk blocks. However, frequent block merging reduces throughput. Thus, we recommend balancing the throughput and disk usage, and setting the threshold according to the needs of applications. Our experiments use a default threshold of 50%.

4) Concurrency control. Fig. 15 compares LUCID with FINE, XIndex, and HPGM under various concurrent workloads. Moreover, we implement a concurrent B+tree variant using per-node locks. LUCID consistently outperforms baselines in all configurations, achieving 19%-115% higher throughput. This is primarily because LUCID utilizes decoupled locks and offers better memory efficiency. From Fig. 15 (a,c) to (b,d), throughput increases across all methods as the number of threads grows, with LUCID exhibiting a steeper improvement. The advantage is especially evident in *insert-heavy* workloads under *zipf* distributions, where LUCID achieves more pronounced performance gains. Fig. 16(a) then evaluates LUCID by varying the number of threads (1 to 16) with different available memory sizes on balanced workloads. As the thread number increases, its effect on throughput diminishes, since disk access becomes the main bottleneck in larger-than-memory databases. The throughput is more sensitive to the thread number when a larger amount of memory is available and with fewer disk accesses, testifying to the reasoning above.

D. Study on Data and Workload Characteristics

1) Record size. We examine the impact of record sizeWe fix the dataset size at 4 GB, allocate 2 GB of memory, and vary record size from 64 B to 2048 B. Fig. 14(b) shows that larger

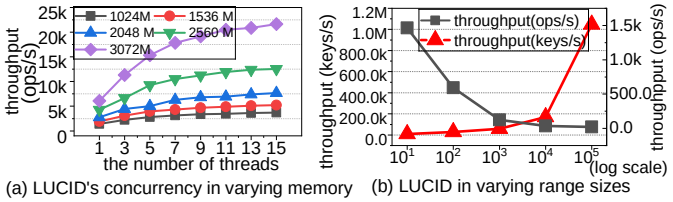

Fig. 16. Varying number of threads and range sizes

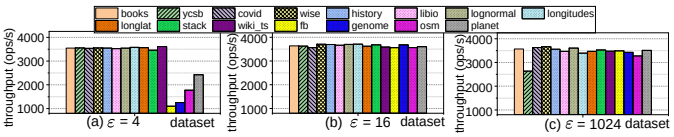

Fig. 17. LUCID performance across various datasets

records have a more pronounced effect on range queries than on point queries. Because blocks (with a fixed block size) hold fewer large records, requiring more blocks to be fetched for a range query, which results in higher I/Os.

2) Range size. We study how the number of keys retrieved per range query influences performance by bulk-loading LUCID on a 4 GB dataset and run a read-only workload, following [11]. Fig. 16(b) shows the results of *wise* as the range size varies from 10 to 10,000. The red triangle curve represents the throughput in *keys/sec*, which increases with larger ranges because a single tree traversal fetches more contiguous records. Conversely, the black square curve shows the throughput in *ops/sec*, which decreases as range sizes grow, since retrieving larger ranges increases disk reads or causes evictions to free memory, slowing each individual operation.

3) Datasets. This experiment examines LUCID's performance on different datasets with varying data distributions. Fig. 17 varies ε for different datasets, as the learned model with different ε shows varying fitting capabilities. We omit $\varepsilon = 32$ and $\varepsilon = 64$ results, as they follow the same trend as $\varepsilon = 16$. On "hard" datasets [11], a very small ε (e.g., 4) creates many leaf nodes, each with limited buffer space, triggering frequent retraining and deeper structures. A very large ε (e.g., 1024) allows bigger nodes but can inflate local search costs in *b_piece*. As discussed in §VI-E1, setting ε between 16 and 64 provides a good balance across diverse datasets.

E. Impacts of Model and Structure Parameters

1) Error bound ε. We study the effect of ε on LUCID. Fig. 18 reports the results for ε spanning 4 to 256 on *wise* dataset. From Fig. 18(a), build time decreases sharply as ε grows from 4 to 32, then stabilizes. In Fig. 18(b), the model size drops as ε increases, because fewer nodes are needed to achieve ε-approximation. Fig. 18(c) shows throughput under different insert/query ratios. As ε exceeds 64, performance declines for *balanced* and *read-heavy* workloads, likely due to larger candidate regions in local searches. Insert-heavy workloads see a sharper decline beyond ε = 16, because a larger *b_piece* can involve more position shifts. Overall, $\varepsilon \in [16, 64]$ works well, with $\varepsilon = 16$ performing best in our tests.

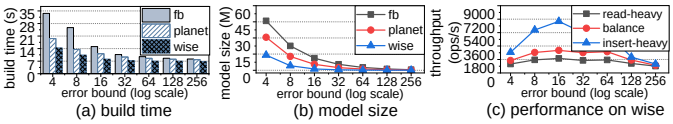

Fig. 18. Performance with different ε in larger-than-memory environments

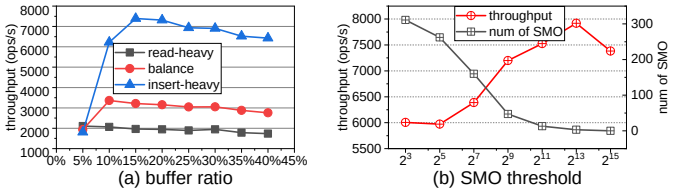

Fig. 19. Impacts of *buffer_ratio*s and SMO thresholds

2) Buffer_ratio. Fig. 19(a) examines how *buffer_ratio* (the pre-allocated buffer space in *b_piece*) affects LUCID under varying insert/query ratios. Throughput of *insert-heavy* workloads first increases as *buffer_ratio* grows, as the insertion benefits from the pre-allocated buffer space. However, performance drops once *buffer_ratio* exceeds 20%. Since a larger buffer size consumes more memory and more data are evicted to disk. For *read-heavy* and *balanced* workloads, throughput declines earlier as *buffer_ratio* increases, as they involve more disk accesses and a larger *buffer_ratio* further exacerbates data eviction. Our experiments set *buffer_ratio* to 30%, in line with the setting in ALEX and B+tree [5] for a fair comparison.

3) SMO thresholds. We vary the SMO threshold, which triggers internal-level retraining after leaf retraining. We run an insert-only workload that frequently fills leaf nodes, prompting *m_piece* creation. Fig. 19(b) shows throughput changes with SMO thresholds from 2^3 to 2^{15}. Lower thresholds ($<2^6$) cause frequent SMOs and lower throughput, while higher thresholds reduce SMOs but risk long *m_piece* chains that slow lookups. A threshold around 2^{10} offers a practical balance.

Parameter guidelines. Based on our experiments, we offer the following guidelines. 1) Use a 64KB block size for workloads dominated by range queries or point queries on larger records; otherwise, use smaller blocks (4-32KB). 2) Apply a buffer ratio of 5% for read-heavy workloads. As the proportion of insertions grows, increase the buffer ratio as needed, but keep it below 30%. 3) More insertions favor a larger SMO threshold (2^{11}-2^{13}), but buffer ratios impose limits. For smaller buffer ratios in insert-heavy workloads, a smaller SMO threshold (2^{10}) is recommended. 4) Adopt a large block merge threshold (80%), if disk resources permit; otherwise, 50% balances performance and disk usage. 5) For disk-intensive workloads, <5 threads are sufficient. More threads would waste CPU without improving performance.

F. Ablation study

We conduct ablation studies by removing LUCID's components and measuring the performance. As depicted in Tab. III, excluding the FS (w/o FS) significantly degrades throughput across all insert/query ratios. The benefit of model-guided buffering (*b_piece*) is evident when comparing LUCID with FILM, which employs a single binary-search buffer (Fig. 7). The impact of the concurrency design has been evaluated in §VI-C4. These results confirm that all individual components are indispensable to LUCID's performance gains.

Tab. III
ABLATION STUDY FOR FS IN LUCID WITH ZIPF WORKLOAD ON WISE

throughput (ops/s)	read-heavy	balance	insert-heavy
w/o FS	1614.17	1053.62	1202.22
LUCID	1942.91	3052.44	6902.19

VII. Related work

Learned indexes. Several benchmarks [11, 13, 40–43] conduct empirical studies to evaluate the performance of learned indexes. Research spans in-memory [3–6, 12, 17–19, 26, 44], and out-of-memory [8, 9, 16, 29, 43] settings. FILM [2] targets larger-than-memory databases with hybrid storage.

Existing dynamic solutions typically rely on preallocated space (e.g., gapped arrays [5, 7, 16, 18] or buffers [4, 6, 44]) and logarithmic merge strategy [17]. However, they either cause interference between existing and newly inserted keys, consume large memory, or underutilize the model-guided optimization in buffers. Several approaches [3, 4, 11, 16] support concurrent operations, often using optimistic locking with version control. TreeLine [10] employs node-level and page-level locks to support concurrency on NVMe SSDs. LOFT [12] proposes a lock-free method for concurrency. However, the Compare-and-Swap primitive used in LOFT is optimized for basic data types (typically $\leq$ 8 bytes) and does not scale well to large records [45], which are common in larger-than-memory settings [2, 20], rendering LOFT unsuitable.

Larger-than-memory databases. Main memory databases offer faster queries than disk-resident ones [1]. But their effectiveness is constrained by memory capacity. Advanced approaches blend the low latency of memory with the large capacity of disk, enabling efficient management of data that exceed memory [2, 10, 20]. Treeline [10] employs record-level caching and pins its index in memory, particularly optimized for SSDs. However, its in-place update strategy makes Treeline less suitable for disk-based settings. Anti-caching [20] expands main-memory DBMS capacity by integrating disk, while maintaining high throughout [1, 21]. FILM [2] is a larger-than-memory learned index based on anti-caching, designed for append-only insertions and single-threaded access.

VIII. Conclusion and Future Work

We propose LUCID, an updatable and concurrent learned index for larger-than-memory data management with dynamic and concurrent workloads. Experiments show that LUCID improves update and query performance, achieves efficient concurrency, and reduces storage overhead.

Acknowledgment

The project was supported by Science Foundation of Hebei Normal University (No. L2026B34), Natural Sciences and Engineering Research Council of Canada (NSERC), and National Natural Science Foundation of China (No.62172423).

AI-Generated Content Acknowledgment

The authors confirm that no AI-generated content was used in the creation of the technical content of this paper. Only revision and proofreading tools were employed to improve grammar and readability, without altering the technical content, ideas, or conclusions.

References

[1] L. Ma, J. Arulraj, S. Zhao, A. Pavlo, S. R. Dulloor, M. J. Giardino, J. Parkhurst, J. L. Gardner, K. Doshi, and S. Zdonik, "Larger-than-memory data management on modern storage hardware for in-memory oltp database systems," in *Workshop on Data Management on New Hardware*, 2016.

[2] C. Ma, X. Yu, Y. Li, X. Meng, and A. Maoliniyazi, "FILM: a fully learned index for larger-than-memory databases," *Proc. VLDB Endow.*, vol. 16, no. 3, pp. 561–573, 2022.

[3] C. Tang, Y. Wang, Z. Dong, G. Hu, Z. Wang, M. Wang, and H. Chen, "XIndex: a scalable learned index for multicore data storage," in *25th ACM SIGPLAN Symposium on PPoPP*, 2020, pp. 308–320.

[4] P. Li, Y. Hua, J. Jia, and P. Zuo, "FINEdex: a fine-grained learned index scheme for scalable and concurrent memory systems," *Proc. VLDB Endow.*, vol. 15, no. 2, pp. 321–334, 2022.

[5] J. Ding, U. F. Minhas, J. Yu, C. Wang, J. Do, Y. Li, H. Zhang, B. Chandramouli, J. Gehrke, D. Kossmann *et al.*, "ALEX: an updatable adaptive learned index," in *ACM SIGMOD International Conference on Management of Data*, 2020, pp. 969–984.

[6] A. Galakatos, M. Markovitch, C. Binnig, R. Fonseca, and T. Kraska, "FITing-Tree: A data-aware index structure," in *SIGMOD*, 2019, pp. 1189–1206.

[7] J. Wu, Y. Zhang, S. Chen, Y. Chen, J. Wang, and C. Xing, "Updatable learned index with precise positions," *Proc. VLDB Endow.*, vol. 14, no. 8, pp. 1276–1288, 2021.

[8] H. Abu-Libdeh, D. Altinbüken, A. Beutel, E. H. Chi, L. Doshi, T. Kraska, X. Li, A. Ly, and C. Olston, "Learned indexes for a google-scale disk-based database," *NeurIPS Workshop on Machine Learning for Systems*, 2020.

[9] H. Lan, Z. Bao, J. S. Culpepper *et al.*, "A fully on-disk updatable learned index," in *ICDE*, 2024, pp. 4856–4869.

[10] G. X. Yu, M. Markakis, A. Kipf, P.-Å. Larson, U. F. Minhas, and T. Kraska, "Treeline: an update-in-place key-value store for modern storage," *Proceedings of the VLDB Endowment*, vol. 16, no. 1, 2022.

[11] C. Wongkham, B. Lu, C. Liu, Z. Zhong, E. Lo, and T. Wang, "Are updatable learned indexes ready?" *Proc. VLDB Endow.*, vol. 15, no. 11, pp. 3004–3017, 2022.

[12] Y. Mo and Y. Hua, "Loft: A lock-free and adaptive learned index with high scalability for dynamic workloads," in *Proceedings of the Twentieth European Conference on Computer Systems*, 2025, pp. 475–491.

[13] Z. Sun, X. Zhou, and G. Li, "Learned index: A comprehensive experimental evaluation," *Proc. VLDB Endow.*, vol. 16, no. 8, pp. 1992–2004, 2023.

[14] P. E. O'Neil, "The SB-tree an index-sequential structure for high-performance sequential access," *Acta Informatica*, vol. 29, no. 3, pp. 241–265, 1992.

[15] C. Ma, X. Yu, Y. Li, A. Maoliniyazi, and X. Meng, " A Learned Approach to Index Algorithm Selection ," in *2024 IEEE International Conference on Data Mining (ICDM)*, 2024, pp. 311–320.

[16] B. Lu, J. Ding, E. Lo, U. F. Minhas, and T. Wang, "APEX: A high-performance learned index on persistent memory," *Proc. VLDB Endow.*, vol. 15, no. 3, pp. 597–610, 2021.

[17] P. Ferragina and G. Vinciguerra, "The PGM-index: a fully-dynamic compressed learned index with provable worst-case bounds," *Proc. VLDB Endow.*, vol. 13, no. 8, pp. 1162–1175, 2020.

[18] P. Li, H. Lu, R. Zhu, B. Ding, L. Yang, and G. Pan, "Dili: A distribution-driven learned index," *Proc. VLDB Endow.*, vol. 16, no. 9, pp. 2212–2224, 2023. [Online]. Available: https://doi.org/10.14778/3598581.3598593

[19] J. Ge, H. Zhang, B. Shi, Y. Luo, Y. Guo, Y. Chai, Y. Chen, and A. Pan, "Sali: A scalable adaptive learned index framework based on probability models," *Proceedings of the ACM on Management of Data*, vol. 1, no. 4, pp. 1–25, 2023.

[20] J. DeBrabant, A. Pavlo, S. Tu, M. Stonebraker, and S. B. Zdonik, "Anti-caching: A new approach to database management system architecture," *Proc. VLDB Endow.*, vol. 6, no. 14, pp. 1942–1953, 2013.

[21] H. Zhang, D. G. Andersen, A. Pavlo, M. Kaminsky, L. Ma, and R. Shen, "Reducing the storage overhead of main-memory OLTP databases with hybrid indexes," in *SIGMOD*, 2016, pp. 1567–1581.

[22] A. Eldawy, J. Levandoski, and P.-Å. Larson, "Trekking through siberia: Managing cold data in a memory-optimized database," *Proc. VLDB Endow.*, vol. 7, no. 11, pp. 931–942, 2014.

[23] J. Levandoski, P. Larson, and R. Stoica, "Indentifying hot and cold data in a main memory oltp engine," in *ICDE*, 2013.

[24] J. O'Rourke, "An on-line algorithm for fitting straight lines between data ranges," *Communications of the ACM*, vol. 24, no. 9, pp. 574–578, 1981.

[25] H. Lan, Z. Bao, J. S. Culpepper, and R. Borovica-Gajic, "Updatable learned indexes meet disk-resident dbms-from evaluations to design choices," *Proceedings of the ACM on Management of Data*, vol. 1, no. 2, pp. 1–22, 2023.

[26] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis, "The case for learned index structures," in *SIGMOD*, 2018, pp. 489–504.

[27] R. C. Fernandez, P. R. Pietzuch, J. Kreps, N. Narkhede, J. Rao, J. Koshy, D. Lin, C. Riccomini, and G. Wang, "Liquid: Unifying nearline and offline big data integra-

tion," in *CIDR*, 2015.
[28] P. Ferragina, F. Lillo, and G. Vinciguerra, "Why are learned indexes so effective?" in *ICML*, 2020, pp. 3123–3132.
[29] J. Zhang, K. Su, and H. Zhang, "Making in-memory learned indexes efficient on disk," *Proc. ACM Manag. Data*, vol. 2, no. 3, p. 151, 2024.
[30] T. Bingmann, 2007, https://panthema.net/2007/stx-btree/.
[31] J. Ding, U. F. Minhas *et al.* (2020) ALEX. [Online]. Available: https://github.com/microsoft/ALEX
[32] P. Li, Y. Hua *et al.* (2021) FINEdex. [Online]. Available: https://github.com/iotlpf/FINEdex
[33] C. Ma, X. Yu, Y. Li *et al.* (2023) FILM. [Online]. Available: https://github.com/chaohcc/film
[34] Y. W. Chuzhe Tang *et al.*, "Xindex," https://ipads.se.sjtu.edu.cn:1312/opensource/xindex.git.
[35] H. Lan, Z. Bao, J. S. Culpepper *et al.*, "Aulid," https://github.com/rmitbggroup/AULID.
[36] G. Yu, M. Markakis, A. Kipf, and Others, "Treeline," https://github.com/mitdbg/treeline.
[37] J. Zhang *et al.*, "Efficient-disk-learned-index public," https://github.com/embryo-labs/Efficient-Disk-Learned-Index.
[38] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with ycsb," in *Proceedings of the 1st ACM symposium on Cloud computing*, 2010, pp. 143–154.
[39] C. Ma, X. Yu, A. M. Yifan Li, and X. Meng, "Lucid," https://github.com/chaohcc/LUCID.
[40] J. Ge, B. Shi, Y. Chai, Y. Luo, Y. Guo *et al.*, "Cutting learned index into pieces: An in-depth inquiry into updatable learned indexes," in *ICDE*, 2023, pp. 315–327.
[41] R. Marcus, A. Kipf, A. van Renen, M. Stoian, S. Misra, A. Kemper, T. Neumann, and T. Kraska, "Benchmarking learned indexes," *Proc. VLDB Endow.*, vol. 14, no. 1, pp. 1–13, 2020.
[42] A. Kipf, R. Marcus, A. van Renen, M. Stoian, A. Kemper, T. Kraska, and T. Neumann, "Sosd: A benchmark for learned indexes," *NeurIPS Workshop on Machine Learning for Systems*, 2019.
[43] H. Lan, Z. Bao, J. S. Culpepper, and R. Borovica-Gajic, "Updatable learned indexes meet disk-resident dbms - from evaluations to design choices," *Proc. ACM Manag. Data*, vol. 1, no. 2, 2023. [Online]. Available: https://doi.org/10.1145/3589284
[44] J. Zhang and Y. Gao, "CARMI: a cache-aware learned index with a cost-based construction algorithm," *Proc. VLDB Endow.*, vol. 15, no. 11, p. 2679–2691, 2022.
[45] G. team. (2025) Built-in Functions for Memory Model Aware Atomic Operations. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc_005f_005fatomic-Builtins.html